

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

HopeLine: Smart Hospital Resource and Blood Donor Matching System

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>CodePulse</i> <i>Team # DSCPP-III-</i> <i>2026-T225</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Khanna, Shivansh –</i> <i>2510012154</i> <i>khannashivansh03@gmail.</i> <i>com</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Srivastava, Piyush –</i> <i>2510370317</i> <i>piyushsrivastava7143@gm</i> <i>ail.com</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Thakur, Parikshit –</i> <i>2510380241</i> <i>parikshitthakur20@gmail.</i> <i>com</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

During medical emergencies, hospitals often face two critical challenges at the same time: finding an available bed, ICU unit or oxygen supply for an incoming patient, and locating a compatible blood donor when a transfusion is urgently needed. Managing these resources manually or on a first-come-first-served basis can cause delays, particularly when several critical patients arrive simultaneously. HopeLine aims to provide an integrated decision-support system that helps prioritize emergency patients and efficiently allocate hospital resources and blood. Each patient is assigned a priority score based on factors such as emergency severity, waiting time, and risk level. The system uses this priority to determine which patient should be considered first when a required resource becomes available.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Currently, most hospitals handle bed allocation and blood donor searches as two separate, largely manual processes. Bed availability is often tracked using physical registers or basic spreadsheet based hospital management software that lists occupancy but does not automatically rank or prioritise patients by urgency. For blood donor matching, hospitals typically rely on static blood bank registries or call volunteer donors listed by blood group without any real-time ranking based on compatibility, distance, availability, or urgency. Currently, there is no widely used system that combines both bed allocation with automated donor matching in one integrated tool.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

General Goals:

- Build a C++ console-based system that models hospitals, patients, beds/resources, and blood donors using Object-Oriented Programming principles.
- Automatically calculate a priority score for each patient based on severity, waiting time, and risk factors, and use it to allocate beds/ICU/oxygen units fairly and quickly.
- Automatically search and rank compatible blood donors using a matching score (compatibility, availability, distance, urgency) whenever a patient requires blood.
- Demonstrate efficient use of data structures (Hash Table, Priority Queue, Queue) to make lookups and ranking fast even as the number of patients/donors grows.

Milestones:

- **Milestone 1:** Design class structure (Patient, Resource, Donor, Hospital) and finalise priority/matching score formulas.
- **Milestone 2:** Implement Hash Table-based lookup for resources and donors by type/blood group.
- **Milestone 3:** Implement Priority Queue logic for both bed allocation (by patient severity) and donor ranking (by matching score).
- **Milestone 4:** Implement overflow Queue for patients/requests when no bed or donor is immediately available.
- **Milestone 5:** Integrate all modules into a single console-based simulation with sample data and test edge cases (no donor found, no bed available).

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

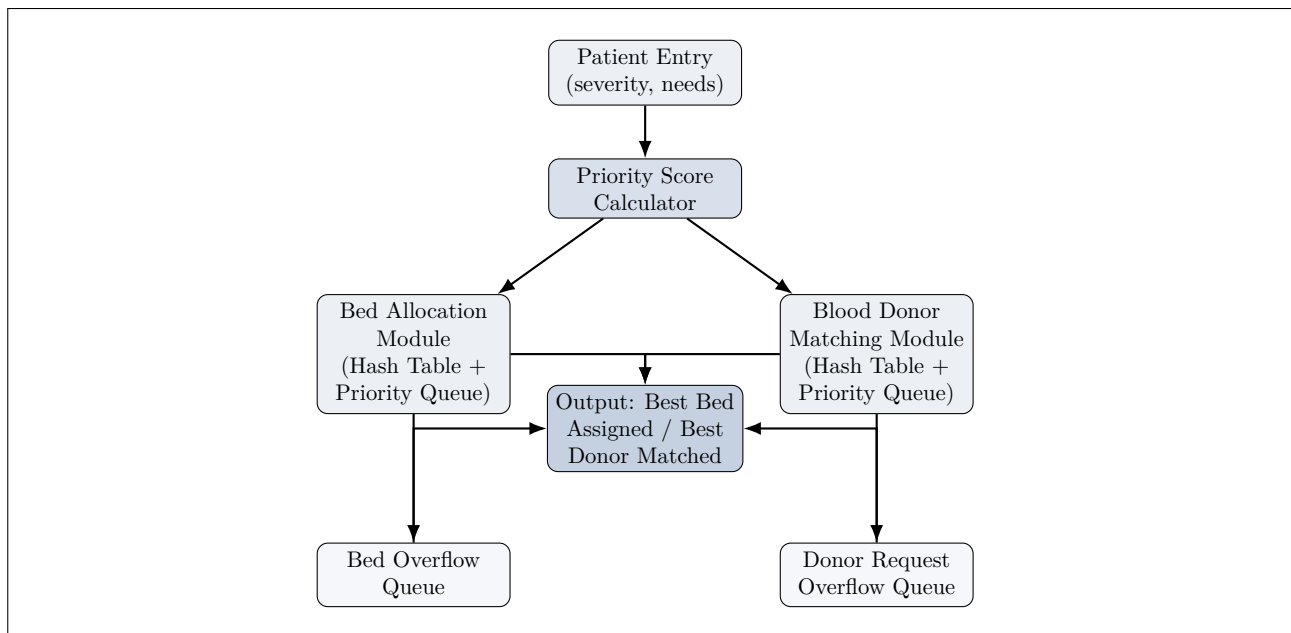
HopeLine will be implemented as a modular C++ application using object-oriented programming and data structures. The system will maintain information about patients, hospital resources, blood units, donors, and emergency requests.

A priority queue will be used to maintain emergency patients according to their calculated priority. A normal queue will manage patients waiting for unavailable resources. Hash tables will provide fast lookup of patients, donors, blood units, and resources. Linked lists will maintain histories such as patient-resource allocation and blood-unit movement.

The C++ implementation will use classes, inheritance, polymorphism, constructors, exception handling, file handling, and STL where appropriate.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The final outcome will be a functional C++ prototype of HopeLine capable of managing emergency patients, hospital resources, blood inventory, and registered donors through a unified system.

The system will provide:

1. Emergency patient registration and priority calculation.
2. Allocation of ICU beds, general beds, oxygen, and other resources.
3. Waiting queues when resources are unavailable.
4. Automatic reassessment and allocation when resources become free.
5. Blood inventory tracking and shortage detection.
6. Registered donor management and compatible donor matching.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

We assume a single hospital for the core implementation, with a fixed, predefined number of beds, ICU units, and oxygen units. Donor data (blood group, availability, last donation date, distance) is assumed to be pre-loaded/mock data rather than pulled from a live database or real donor registry. We also assume standard blood group compatibility rules and a fixed minimum gap (90 days) between donations for donor eligibility. Real-time GPS-based distance calculation is out of scope; a simplified distance value will be used instead.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

- GeeksforGeeks – Priority Queue, Hash Table, and Graph/Dijkstra’s Algorithm documentation (<https://www.geeksforgeeks.org>)
- Ministry of Health & Family Welfare – e-RaktKosh (<https://eraktkosh.mohfw.gov.in/>)
- C++ Reference (<https://en.cppreference.com/>)